BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

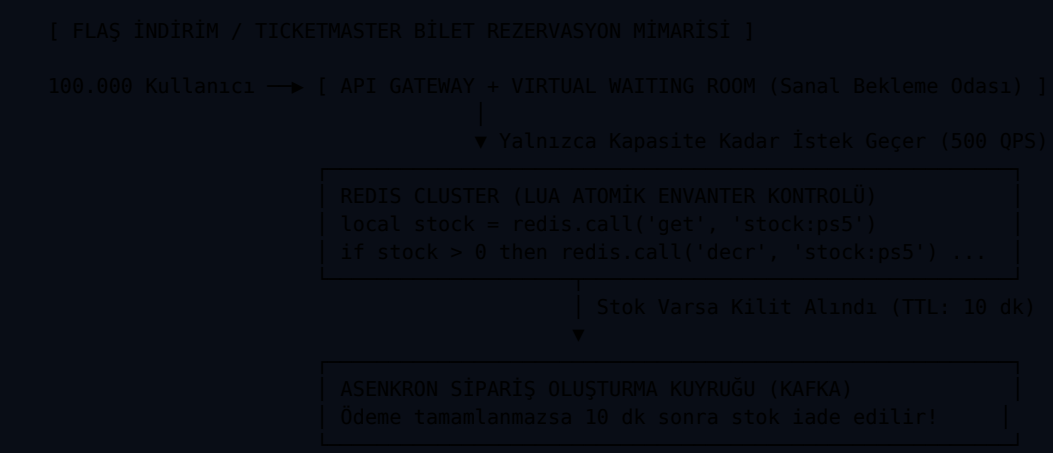
48

E-Ticaret Flaş İndirim ve Bilet Satış Sistemi (Flash Sale / Ticketmaster)

Designing a Flash Sale & Ticketing System (Ticketmaster / Amazon Prime Day)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Stadyumda bir rock konseri için sadece 1.000 bilet olduğunu ve kapıda 100.000 kişinin beklediğini düşünün. Kapıyı ardına kadar açarsanız 100.000 kişi içeri dalar, gişeyi kırar ve izdihamda herkes ezilir (Veritabanı çöküşü!). Çözüm: 1) Kapının önüne döner bir bariyer (Virtual Waiting Room) koyarsınız. İçeriye sadece sırayla 1.000 kişi alınır. 2) Gişedeki biletçi sepete eklenen bilete 10 dakikalık bir geri sayım sayacı yapıştırır (Geçici Kilit). 10 dakika içinde parayı ödemezseniz bilet elinizden alınır ve sıradaki kişiye verilir!

🇬🇧 ENGLISH (ELI5)

Picture a stadium rock concert with only 1,000 tickets, but a mob of 100,000 fans waiting outside. If you throw open the gates, 100,000 people trample the box office and smash the counters (Database meltdown!). The solution: 1) Set up a velvet rope turnstile outside (Virtual Waiting Room). Fans queue in order; only batches of 500 enter at a time. 2) When a fan grabs a seat, the system slaps a 10-minute countdown timer on the ticket (Atomic Reservation with TTL). If they don't enter credit card info within 10 minutes, the hold expires and the seat returns to the pool!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Flaş İndirim Mimarisi: 1) Sanal Bekleme Odası (Virtual Waiting Room): Cloudflare Waiting Room veya Kafka kuyrukları ile kullanıcılar adil bir sıra numarası (FIFO) ile bekletilir; veritabanı kapasitesi kadar kullanıcı içeri alınır. 2) Bellek İçi Atomik Envanter (In-Memory Atomic Inventory): Veritabanına `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0` yapmak satır kilidi (Row Lock) yüzünden DB'yi kilitler. Stok Redis'te tutulur ve Lua Script ile atomik olarak düşürülür (`DECR`). Sıfırın altına düşmesine asla izin verilmez. 3) Geçici Rezervasyon ve Dağıtık TTL: Kullanıcı bilet seçtiğinde bilet 10 dakika rezerve edilir (`SET seat:42:lock user_101 EX 600`). Ödeme başarılı olursa DB'ye kesin satış yazılır. Ödeme başarısız olursa veya 10 dakika dolarsa TTL biter ve bilet havuza geri döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Flash Sale & Ticketing Architecture: 1) Virtual Waiting Room (Fair Queuing): Deploys an edge ingress queue (Cloudflare Waiting Room) metering traffic based on backend throughput capacity, preventing database saturation. 2) Atomic In-Memory Stock Depletion: Executing relational database updates (`UPDATE stock SET qty = qty - 1 WHERE qty > 0`) creates severe row-level lock contention. Inventory is decremented in-memory inside Redis via atomic Lua scripts in microsecond speeds. 3) Two-Phase Inventory Hold (Reservation TTL): Stock is not permanently sold until payment commits. Grabbing an item creates a temporary hold with a 10-minute expiration (`SET item:hold:user EX 600`). If payment confirmation completes, a Kafka event finalizes the database commit; if the timer expires, the hold releases atomically back to available inventory.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Flaş İndirim Atomik Stok ve Geçici Rezervasyon Motoru
import java.util.concurrent.*;

public class FlashSaleEngine {
    private int availableStock;
    private final ConcurrentMap<String, Long> activeReservations = new ConcurrentHashMap<>();

    public FlashSaleEngine(int initialStock) {
        this.availableStock = initialStock;
    }

    public synchronized boolean tryReserveItem(String userId, long holdDurationMillis) {
        if (availableStock <= 0) {
            IO.println("❌ STOK TÜKENDİ! Kullanıcı: " + userId);
            return false; // Stok bitti
        }

        availableStock--;
        activeReservations.put(userId, System.currentTimeMillis() + holdDurationMillis);
        IO.println("✔ Stok Rezerve Edildi: " + userId + " (Kalan Stok: " + availableStock + ")");
        return true;
    }

    public synchronized void cancelExpiredReservation(String userId) {
        Long expiration = activeReservations.remove(userId);
        if (expiration != null) {
            availableStock++; // Stok havuza iade edildi
            IO.println("⏱ Süre Doldu! Stok İade Edildi: " + userId + " (Güncel Stok: " +
availableStock + ")");
        }
    }
}

void main() {
    var sale = new FlashSaleEngine(2); // Toplam 2 adet PS5
    sale.tryReserveItem("user_ali", 600_000);
    sale.tryReserveItem("user_veli", 600_000);
    sale.tryReserveItem("user_can", 600_000); // Stok kalmadı!

    sale.cancelExpiredReservation("user_ali"); // Ali ödemedi -> Stok iade!
    sale.tryReserveItem("user_can", 600_000);  // Can artık alabildi!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Stoku doğrudan MySQL'de kilit açarak (SELECT FOR UPDATE) yönetirim' demeyin! Saniyede 100.000 kişinin aynı satırı kilitlemeye çalışması veritabanında Connection Pool tükenmesine ve sistemin çökmesine neden olur. Stok kontrolü daima Redis'te atomik yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch relational `SELECT ... FOR UPDATE` row locks for flash sales. 50,000 concurrent threads vying for the exact same database row lock will crash the connection pool in seconds. Manage hot stock counts in Redis memory via atomic Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal Bekleme Odası (Virtual Waiting Room) veritabanını aşırı yükten korur.

• Stok düşürme işlemi veritabanı yerine Redis üzerinde atomik Lua scripti ile yapılır.

• Geçici rezervasyon ve TTL sayesinde ödeme yapmayan kullanıcıların ürünleri havuza döner.

🎯 Key Takeaways for the Interview

• Virtual waiting rooms throttle user concurrency to match backend database processing limits.

• Atomic in-memory decrements in Redis eliminate relational database row-locking contention.

• Reservation TTL windows release unclaimed inventory automatically back to waiting fans.